Practical No.7: Implement program to identify the most important features that contribute to the model accuracy

X. Conclusion

Random Forest model was used to find which features are most important for prediction.
It showed that some features affect the result more than others.
By selecting only important features, the model becomes more accurate, faster, and easy to understand.

XI. Practical Related Questions

1. What is data preprocessing and why is it important?

Data preprocessing is the process of cleaning and preparing raw data before using it in a model.
It is important because clean data gives better accuracy and performance.

---

2. How do you handle missing values?

Common methods:

- Remove rows with missing values
- Fill with mean/median/mode
- Use interpolation or prediction methods

---

3. Methods to determine feature importance

- Filter methods (correlation, ANOVA)
- Wrapper methods (RFE)
- Embedded methods (Random Forest, Lasso)

---

4. How does correlation help?

Correlation shows relationship between feature and target.
Higher correlation = more important feature.

---

5. How Random Forest finds feature importance?

Random Forest checks how much each feature reduces error in decision trees.
Features that reduce error more are considered more important.

6. Write a Python program to identify and visualize the most important features that contribute to model accuracy.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

# Simple dataset (Employee Performance)
data = {
    'Experience': [1, 2, 3, 4, 5, 6, 7, 8],
    'Projects': [1, 2, 2, 3, 3, 4, 4, 5],
    'TrainingHours': [10, 15, 20, 25, 30, 35, 40, 45],
    'Promoted': [0, 0, 0, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

# Features and target
X = df[['Experience', 'Projects', 'TrainingHours']]
y = df['Promoted']

# Train model
model = RandomForestClassifier(n_estimators=50, random_state=0)
model.fit(X, y)

# Feature importance
importance = model.feature_importances_

# Display importance
print("Feature Importance:")
for i, col in enumerate(X.columns):
    print(col, ":", importance[i])

# Visualization
plt.bar(X.columns, importance)
plt.title("Feature Importance (Employee Promotion)")
plt.xlabel("Features")
plt.ylabel("Importance Score")
plt.show()
```

Output

```
Feature Importance:
Experience : 0.3541666666666667
Projects : 0.375
TrainingHours : 0.2708333333333333
```

- **Experience** → High importance (main factor for promotion)
- **Projects** → Medium importance
- **TrainingHours** → Less importance

This shows **which employee factor affects promotion most**

7. How feature importance varies in algorithms?

- Decision Tree → Based on splits
- Random Forest → Average of many trees
- Linear Regression → Based on coefficients
- SVM → Based on support vectors

Different algorithms calculate importance differently.

XII Exercise Answers

1. Implement feature engineering techniques to prepare data for machine learning models.

Feature Engineering is the process of selecting, cleaning, transforming, and creating useful features from raw data to improve the performance of a machine learning model.

Feature Engineering Steps

- Handle missing values
- Encode categorical data
- Normalize/scale data
- Remove irrelevant features

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# Step 1: Create simple dataset
data = {
    'Name': ['Amit', 'Riya', 'Sohan', 'Neha', 'Raj'],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],
    'Age': [20, 21, None, 22, 23],
    'Marks': [45, 50, 55, None, 65],
    'City': ['Mumbai', 'Pune', 'Mumbai', 'Delhi', 'Pune']
}

df = pd.DataFrame(data)

print("Original Dataset:\n", df)

# --------------------------------
# Step 2: Handle Missing Values
# --------------------------------
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

# --------------------------------
# Step 3: Encode Categorical Data
# --------------------------------
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
df['City'] = le.fit_transform(df['City'])

# --------------------------------
# Step 4: Normalize / Scale Data
# --------------------------------
scaler = MinMaxScaler()
df[['Age', 'Marks']] = scaler.fit_transform(df[['Age', 'Marks']])

# --------------------------------
# Step 5: Remove Irrelevant Feature
# --------------------------------
df.drop('Name', axis=1, inplace=True)

# Final dataset
print("\nAfter Feature Engineering:\n", df)
```

Output

```
Original Dataset:
     Name  Gender   Age  Marks    City
0   Amit    Male  20.0   45.0  Mumbai
1   Riya  Female  21.0   50.0    Pune
2  Sohan    Male   NaN   55.0  Mumbai
3   Neha  Female  22.0    NaN   Delhi
4    Raj    Male  23.0   65.0    Pune

After Feature Engineering:
    Gender       Age   Marks  City
0       1  0.000000  0.0000     1
1       0  0.333333  0.2500     2
2       1  0.500000  0.5000     1
3       0  0.666667  0.4375     0
4       1  1.000000  1.0000     2
```

After feature engineering:

- **Missing values** in Age and Marks were filled
- **Gender and City** were converted into numbers (encoding)
- **Age and Marks** were scaled between **0 and 1**

Result: Data is now **clean, numeric, and ready for machine learning model**

2. Write a program to identify the most important features that contributes to the model accuracy.

```python
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Updated Dataset (StudyHours strongly affects Result)
data = {
    'StudyHours': [1,2,3,6,7,8,9,10],
    'SleepHours': [6,6,7,6,7,6,7,6],
    'Attendance': [60,65,70,72,75,78,80,82],
    'Result': [0,0,0,1,1,1,1,1]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['StudyHours','SleepHours','Attendance']]
y = df['Result']

# Train Random Forest Model
model = RandomForestClassifier(n_estimators=50, random_state=0)
model.fit(X, y)

# Display Feature Importance
print("Feature Importance:\n")
for name, score in zip(X.columns, model.feature_importances_):
    print(name, ":", score)
```

Output

```
Feature Importance:

StudyHours : 0.503670634920635
SleepHours : 0.04174272486772488
Attendance : 0.45458664021164014
```

- **StudyHours (0.50)** → Most important factor for passing
- **Attendance (0.45)** → Second important factor
- **SleepHours (0.04)** → Very less impact